300 DSA Problems

For Neuroplasticity, Critical Thinking & Creativity

A Brain-Rewiring Problem-Solving Journey

From Beginner to Advanced - Topic Wise
With Problem Origins, Core Concepts, Approach Patterns
& Mindfulness Framework for When You Are Stuck

"Your brain physically rewires itself every time you struggle with a hard problem.
The struggle IS the growth."

PART 1: THE NEUROSCIENCE OF LEARNING DSA

Why Neuroplasticity Matters for DSA

Research from Oxford University (2023) found that learning programming causes measurable structural changes in 8 distinct brain regions - including the frontal pole (persistence and goal achievement), medial frontal gyrus (deductive reasoning and decision making), bilateral pallidum (working memory and reinforcement learning), and the cerebellum (pattern coordination). This means every time you solve a DSA problem, you are literally building new neural architecture.

JHU researchers discovered that coding activates the brain's logical reasoning centers in the left hemisphere, overlapping with language processing areas. This means DSA practice strengthens the same circuits used for logical argumentation, mathematical reasoning, and structured communication - skills that transfer far beyond programming.

How Your Brain Grows Through DSA Problems

1. Synaptic Strengthening (Repetition): Each time you practice a pattern like two-pointer or sliding window, the neural pathways for that pattern become physically stronger and faster. This is why spaced repetition works - the brain consolidates during sleep.

2. Myelination (Speed): With practice, the brain wraps myelin sheaths around frequently-used neural pathways, making signal transmission up to 100x faster. This is why experienced problem-solvers 'see' solutions instantly - their pattern-matching circuits are literally insulated for speed.

3. Pruning (Efficiency): The brain prunes neural connections that are rarely used. By practicing diverse problem types, you ensure a rich set of thinking tools survives pruning. This is why variety in problem difficulty and topic is essential.

4. Dopamine Loops (Motivation): Each solved problem releases dopamine, reinforcing the neural circuits that led to the solution. This reward-based learning creates a positive feedback loop that makes you increasingly effective and motivated.

The 90-Day Neuroplastic DSA Plan

Based on neuroplasticity research, here is the optimal schedule for 300 problems over ~90 days. The brain needs sleep to consolidate learning, so never exceed 4-5 new problems per day. Quality of struggle matters more than quantity.

Phase	Weeks	Problems/Day	Focus	Brain Region Activated
Foundation	1-3	3-4 Easy	Arrays, Strings, Hash Maps	Pallidum (working memory)
Pattern Building	4-6	3-4 Easy/Med	Two Pointers, Sliding Window, Stacks	Frontal pole (persistence)
Deep Reasoning	7-9	3 Medium	Trees, Graphs, Recursion	Medial frontal (deduction)
Integration	10-11	2-3 Med/Hard	DP, Advanced Graphs, Tries	Prefrontal (executive control)
Mastery	12-13	2-3 Hard	Mixed topics, Contests	Full network integration

PART 2: THE MINDFUL PROBLEM-SOLVER'S GUIDE

"Frustration is not the enemy of learning - it IS learning. Your brain is rewiring in real time."

What TO DO When You Cannot Solve a Problem

Step 1: Breathe and Acknowledge (2 minutes)

Close your eyes. Take 5 deep breaths (4 counts in, 6 counts out). Say to yourself: 'I am stuck, and that is perfectly normal. My brain is working on building new connections right now.' This is not woo-woo - research shows that acknowledging frustration reduces cortisol (stress hormone) and allows the prefrontal cortex to re-engage with the problem.

Step 2: Re-read the Problem with Beginner's Mind (5 minutes)

Pretend you have never seen this problem before. Read every word carefully. Draw the input/output. Write down what you KNOW and what you DON'T KNOW. Often we are stuck because we made an assumption in the first 30 seconds that locked us into a wrong path.

Step 3: Shrink the Problem (5 minutes)

Can you solve it for n=1? n=2? n=3? Write out small examples by hand. Pattern recognition is your brain's visual cortex at work. Many breakthroughs come from manually tracing tiny inputs.

Step 4: Name the Category (2 minutes)

Ask: Does this look like a sliding window? A graph traversal? A DP subproblem? Just naming the possible category activates your brain's associative memory and pulls relevant patterns.

Step 5: Write Pseudocode or Brute Force (10 minutes)

A brute force solution is ALWAYS valuable. It proves you understand the problem, gives you a correct reference to test against, and often reveals the optimization path.

Step 6: Take a Timed Break (15-20 minutes)

Walk, stretch, or do something unrelated. Your brain continues working on the problem unconsciously - this is called 'incubation' and is well-documented in creativity research. Many 'shower thoughts' are actually your default mode network solving problems in the background.

Step 7: Study the Solution Mindfully (15 minutes)

If after 45 minutes of genuine effort you are still stuck, read the solution. But do NOT just passively read it. Ask: WHY does this approach work? WHAT pattern does it use? HOW would I recognize a similar problem? Then close the solution and re-implement from memory.

What NOT TO DO When Stuck

â■¢ DO NOT stare at the screen for hours without writing anything. After 5 minutes of no progress, switch strategies.

â■¢ DO NOT immediately look at the solution. Give yourself at least 25-30 minutes of genuine struggle. The struggle builds neural pathways.

â■¢ DO NOT tell yourself 'I am not smart enough.' Intelligence in DSA is built, not born. Your brain literally grows from each attempt.

â■¢ DO NOT skip the problem and never return. Mark it, move on, and revisit in 2-3 days. Spaced repetition is neuroplasticity's best friend.

â■¢ DO NOT compare your speed to others. Everyone's neural wiring is different. Your 3-hour problem today becomes a 10-minute pattern next month.

â■¢ DO NOT solve the same easy problems repeatedly for comfort. Growth requires operating at the edge of your ability (the 'zone of proximal development').

â■¢ DO NOT code without thinking first. Jumping to code before understanding the problem trains bad neural patterns.

â■¢ DO NOT lose sleep over DSA. Sleep is when your brain consolidates learning into long-term memory. 7-8 hours is non-negotiable.

â■¢ DO NOT study in marathon sessions. Research shows 90-minute focused blocks with breaks are optimal for neuroplastic change.

â■¢ DO NOT be ashamed of reading editorials. Even experts learned from others. The key is HOW you read - actively, not passively.

MINDFULNESS CHECKPOINT: Before every problem-solving session, rate your mental state 1-10. If below 5, do 5 minutes of breathing first. A calm brain solves problems 40% faster than an anxious one.

PART 3: 300 DSA PROBLEMS - TOPIC WISE

Each problem includes its historical origin or why it exists, the core concepts needed, and the approach pattern to develop. Problems progress from beginner to advanced within each topic.

TOPIC 1: ARRAYS & HASHING (Problems 1-35)

Arrays are the foundational data structure. Hashing transforms $O(n^2)$ brute-force searches into O(n) lookups. Mastering these builds the pallidum (working memory) and frontal pole (pattern matching).

Beginner (Build Working Memory)

[Easy] #1. Two Sum

Origin: One of the earliest interview problems (Google ~2010). Tests if you can trade space for time.

Key Concepts: Hash map, complement lookup

Approach: Brute force O(n2) first, then optimize with hash map storing complement.

[Easy] #2. Contains Duplicate

Origin: Classic set membership test. Origin: database duplicate detection.

Key Concepts: Hash set, early termination

Approach: Insert into set; if already present, return true. O(n) time, O(n) space.

[Easy] #3. Valid Anagram

Origin: From cryptography and word games. Frequency counting is foundational.

Key Concepts: Character frequency, hash map or array[26]

Approach: Count chars in both strings, compare frequency arrays.

[Easy] #4. Best Time to Buy and Sell Stock

Origin: Real-world finance problem. Kadane's insight applied to prices.

Key Concepts: Running minimum, max profit tracking

Approach: Track min price so far; at each step compute profit vs. best profit.

[Easy] #5. Maximum Subarray (Kadane's)

Origin: Discovered by Jay Kadane (1984) at CMU. Elegant DP insight.

Key Concepts: Dynamic programming, local vs global optimum

Approach: At each element: extend current subarray or start fresh. Track global max.

[Easy] #6. Merge Sorted Array

Origin: From merge sort's merge step. Practical in database operations.

Key Concepts: Two pointers, in-place merge from the end

Approach: Fill from the back of the array to avoid overwriting.

[Easy] #7. Move Zeroes

Origin: Data compaction problem from systems programming.

Key Concepts: Two pointers (read/write), in-place

Approach: Write pointer tracks next non-zero position; swap as you read.

[Easy] #8. Plus One

Origin: Simulates arbitrary-precision arithmetic. Origin: big number libraries.

Key Concepts: Carry propagation, edge case (all 9s)

Approach: Process from least significant digit; handle carry chain.

[Easy] #9. Single Number

Origin: Bit manipulation classic. XOR's self-canceling property.

Key Concepts: XOR, a^a=0, a^0=a

Approach: XOR all elements; pairs cancel, leaving the unique element.

[Easy] #10. Intersection of Two Arrays II

Origin: Set operations from database theory (SQL INTERSECT).

Key Concepts: Hash map frequency, or sorting + two pointers

Approach: Count frequencies in map; iterate second array decrementing counts.

NEUROPLASTICITY NOTE: After completing problems 1-10, your brain has formed initial pathways for hash map lookups and two-pointer patterns. Sleep consolidates these. Review them 48 hours later.

Intermediate (Build Pattern Recognition)

[Medium] #11. Product of Array Except Self

Origin: Amazon interview classic. Tests prefix/suffix decomposition thinking.

Key Concepts: Prefix products, suffix products, no division constraint

Approach: Two passes: forward for prefix products, backward for suffix products.

[Medium] #12. Top K Frequent Elements

Origin: From information retrieval and search ranking systems.

Key Concepts: Hash map + heap (or bucket sort)

Approach: Count frequencies, then use min-heap of size k or bucket sort by frequency.

[Medium] #13. Encode and Decode Strings

Origin: Serialization problem from network protocols (HTTP, Protocol Buffers).

Key Concepts: Length-prefix encoding, delimiter design

Approach: Encode as 'length#string' for each word. Decode by parsing length then extracting.

[Medium] #14. Longest Consecutive Sequence

Origin: From computational geometry and database indexing.

Key Concepts: Hash set, intelligent sequence start detection

Approach: Only start counting from numbers where (num-1) is NOT in set. O(n) total.

[Medium] #15. Group Anagrams

Origin: Natural language processing and dictionary compression.

Key Concepts: Sorted string as key, or character count tuple as key

Approach: Map sorted(word) -> list of anagrams. Or use frequency tuple as key.

[Medium] #16. 3Sum

Origin: Extension of Two Sum. Classic combinatorial problem from computational geometry.

Key Concepts: Sorting + two pointers, duplicate skipping

Approach: Sort array. Fix one element, use two pointers for remaining pair. Skip duplicates.

[Medium] #17. Container With Most Water

Origin: Geometric optimization. Origin: operations research.

Key Concepts: Two pointers from ends, greedy shrinking

Approach: Start with widest container. Move the shorter line inward (greedy).

[Medium] #18. Subarray Sum Equals K

Origin: Prefix sum technique. Origin: signal processing and cumulative statistics.

Key Concepts: Prefix sum, hash map of prefix sum frequencies

Approach: Track cumulative sum; check if (sum - k) exists in map.

[Medium] #19. Next Permutation

Origin: Combinatorics from Dijkstra's era. Lexicographic ordering.

Key Concepts: Find rightmost ascent, swap, reverse suffix

Approach: Find largest i where a[i] < a[i+1]. Swap a[i] with next larger. Reverse suffix.

[Medium] #20. Rotate Array

Origin: Circular buffer operations from OS and embedded systems.

Key Concepts: Reverse entire array, reverse first k, reverse rest

Approach: Three reverses: whole array, first k elements, remaining elements.

Advanced (Build Executive Control)

[Hard] #21. First Missing Positive

Origin: Database integrity check. Pigeonhole principle applied.

Key Concepts: Index as hash, cyclic sort, O(1) space constraint

Approach: Place each number at index (num-1). First index where a[i] != i+1 is answer.

[Hard] #22. Trapping Rain Water

Origin: Computational geometry classic. Elevation map processing.

Key Concepts: Left max/right max arrays, or two pointers, or monotonic stack

Approach: Water at each position = min(leftMax, rightMax) - height. Two-pointer O(1) space.

[Hard] #23. Median of Two Sorted Arrays

Origin: Database merge operation. Requires binary search insight.

Key Concepts: Binary search on smaller array, partition logic

Approach: Binary search partition on smaller array. Ensure left halves <= right halves.

[Hard] #24. 4Sum

Origin: Generalization of 3Sum. k-Sum framework.

Key Concepts: Sort + recursive k-sum reduction to 2Sum

Approach: Reduce to 3Sum by fixing first element. Reduce 3Sum to 2Sum with two pointers.

[Hard] #25. Sliding Window Maximum

Origin: Stream processing and real-time analytics.

Key Concepts: Monotonic deque, maintaining decreasing order

Approach: Deque stores indices of potential maximums in decreasing value order.

[Easy] #26. Remove Duplicates from Sorted Array

Origin: In-place data cleaning from database ops.

Key Concepts: Two pointers (slow/fast), in-place modification

Approach: Slow pointer marks write position; fast scans for new unique values.

[Easy] #27. Search Insert Position

Origin: Binary search foundation. Origin: interpolation in sorted lists.

Key Concepts: Binary search, leftmost insertion point

Approach: Standard binary search; return lo when not found.

[Medium] #28. Spiral Matrix

Origin: Matrix traversal used in image processing and rasterization.

Key Concepts: Boundary tracking (top, bottom, left, right)

Approach: Traverse in spiral by shrinking boundaries after each direction.

[Medium] #29. Set Matrix Zeroes

Origin: Sparse matrix operations from scientific computing.

Key Concepts: Use first row/col as markers, in-place

Approach: Mark rows/cols to zero using first row and first col as flags.

[Easy] #30. Majority Element (Boyer-Moore)

Origin: Voting algorithm by Boyer and Moore (1981).

Key Concepts: Candidate tracking, count cancellation

Approach: Maintain candidate and count; increment for match, decrement otherwise.

[Easy] #31. Pascal's Triangle

Origin: Ancient mathematical object (China, Persia, Europe). Combinatorics.

Key Concepts: Previous row generates next row

Approach: Each element = sum of two elements above. Build row by row.

[Medium] #32. Jump Game

Origin: Greedy reachability from graph theory.

Key Concepts: Greedy, max reach tracking

Approach: Track farthest reachable index. If current > maxReach, return false.

[Medium] #33. Jump Game II

Origin: Minimum hops problem. BFS on implicit graph.

Key Concepts: BFS level-by-level, or greedy with range tracking

Approach: Track current range end and farthest reach. Increment jumps at range end.

[Medium] #34. Find All Duplicates in Array

Origin: Pigeonhole principle variant. Index marking technique.

Key Concepts: Negate-at-index marking, O(1) space

Approach: For each num, negate arr[abs(num)-1]. If already negative, it's a duplicate.

[Hard] #35. Candy Distribution

Origin: Scheduling and fairness from HR/resource allocation.

Key Concepts: Two-pass greedy: left-to-right then right-to-left

Approach: First pass: satisfy left neighbors. Second pass: satisfy right neighbors. Take max.

TOPIC 2: STRINGS (Problems 36-60)

String problems activate language processing and pattern recognition centers simultaneously. The brain's Broca's area (language production) and Wernicke's area (comprehension) both engage, making strings uniquely powerful for building cross-domain neural connections.

[Easy] #36. Valid Palindrome

Origin: Ancient wordplay (earliest known palindrome: Latin 'sator' square, 79 AD).

Key Concepts: Two pointers, alphanumeric check

Approach: Two pointers from ends, skip non-alphanumeric, compare lowercase.

[Easy] #37. Reverse String

Origin: Foundational string operation from text processing.

Key Concepts: Two pointers, in-place swap

Approach: Swap characters from both ends moving inward.

[Easy] #38. First Unique Character

Origin: Text analysis for compression and search indexing.

Key Concepts: Frequency count with hash map or array[26]

Approach: First pass: count. Second pass: find first with count=1.

[Easy] #39. Valid Parentheses

Origin: Compiler design: expression parsing. Origin: Dijkstra's shunting-yard.

Key Concepts: Stack, matching pairs

Approach: Push open brackets. Pop and match for close brackets. Stack must be empty at end.

[Medium] #40. Implement strStr() (KMP)

Origin: Knuth-Morris-Pratt (1977). Revolutionized string matching.

Key Concepts: Failure function / partial match table, finite automaton

Approach: Build failure table from pattern. Use it to avoid re-scanning matched characters.

[Medium] #41. Longest Substring Without Repeating

Origin: Network packet analysis, unique token windows.

Key Concepts: Sliding window, hash set/map for last position

Approach: Expand right pointer; when duplicate found, shrink left past previous occurrence.

[Medium] #42. Longest Palindromic Substring

Origin: Bioinformatics (DNA palindromes mark restriction enzyme sites).

Key Concepts: Expand around center, or Manacher's algorithm

Approach: Try each position (and gap) as center. Expand outward while palindromic.

[Medium] #43. String to Integer (atoi)

Origin: C standard library function. Parsing and edge case handling.

Key Concepts: State machine, overflow detection

Approach: Skip whitespace, handle sign, accumulate digits, check overflow before multiply.

[Medium] #44. Count and Say

Origin: Recreational math sequence (Morris number sequence, 1986).

Key Concepts: Run-length encoding, simulation

Approach: Iterate previous term, group consecutive identical digits, encode as count+digit.

[Medium] #45. Zigzag Conversion

Origin: Text layout and display formatting.

Key Concepts: Row assignment by modular arithmetic or simulation

Approach: Assign chars to rows using zigzag index pattern. Concatenate rows.

[Easy] #46. Longest Common Prefix

Origin: Dictionary/trie construction, auto-complete systems.

Key Concepts: Vertical scanning, or binary search on prefix length

Approach: Compare character-by-character across all strings at each position.

[Medium] #47. Palindromic Substrings (Count)

Origin: Extension of palindrome detection. Combinatorial counting.

Key Concepts: Expand around center for each of 2n-1 centers

Approach: Count palindromes centered at each char and between each pair.

[Medium] #48. Decode Ways

Origin: Cryptography / encoding interpretation (like A=1, B=2).

Key Concepts: Dynamic programming, digit grouping decisions

Approach: DP[i] = ways considering first i chars. Check 1-digit and 2-digit decodings.

[Medium] #49. Word Break

Origin: Natural language processing: text segmentation (Chinese/Japanese text).

Key Concepts: DP with dictionary lookup, or BFS/Trie

Approach: DP[i] = can s[0..i] be segmented? Check all j < i where DP[j] and s[j..i] in dict.

[Hard] #50. Minimum Window Substring

Origin: Text search and bioinformatics (finding gene subsequences).

Key Concepts: Sliding window with character frequency map

Approach: Expand to include all chars of t. Contract from left while still valid. Track minimum.

[Hard] #51. Edit Distance

Origin: Levenshtein (1965). Spell checkers, DNA sequence alignment.

Key Concepts: 2D DP, insert/delete/replace operations

Approach: DP[i][j] = min edits for s[0..i] to t[0..j]. Three choices at each cell.

[Hard] #52. Regular Expression Matching

Origin: Thompson (1968) and Unix. Foundation of text processing tools.

Key Concepts: 2D DP or recursive with memoization, '.' and '*' handling

Approach: DP[i][j] = does s[0..i] match p[0..j]. Handle '*' as 0 or more of preceding.

[Hard] #53. Wildcard Matching

Origin: File system glob patterns (Unix shell, DOS).

Key Concepts: 2D DP, or greedy with backtracking checkpoint

Approach: DP approach or track last '*' position and backtrack point.

[Medium] #54. Longest Repeating Character Replacement

Origin: Signal processing: longest segment with limited noise.

Key Concepts: Sliding window, track max frequency character count

Approach: Window valid when (window_size - maxFreq) <= k. Expand/contract accordingly.

[Medium] #55. Group Shifted Strings

Origin: Caesar cipher classification from cryptography.

Key Concepts: Normalize shift pattern as key

Approach: Compute difference pattern between consecutive chars (mod 26) as hash key.

[Medium] #56. Reverse Words in a String

Origin: Text formatting and display rendering.

Key Concepts: Split, reverse list, join (or in-place reverse)

Approach: Reverse entire string, then reverse each word individually.

[Medium] #57. Find All Anagrams in String

Origin: Plagiarism detection, genomic pattern matching.

Key Concepts: Sliding window of size p.length, frequency comparison

Approach: Fixed-size window with frequency count. Slide and compare to pattern frequency.

[Medium] #58. Rabin-Karp String Matching

Origin: Rabin & Karp (1987). Rolling hash for multi-pattern search.

Key Concepts: Rolling polynomial hash, modular arithmetic

Approach: Compute hash for window; slide by removing left char and adding right char.

[Hard] #59. Shortest Palindrome (KMP-based)

Origin: Constructive string problem using failure function.

Key Concepts: KMP failure function on s + '#' + reverse(s)

Approach: Find longest palindromic prefix using KMP. Prepend missing suffix reversed.

[Hard] #60. Text Justification

Origin: Typesetting algorithm (Knuth's TeX, 1978). Real-world formatting.

Key Concepts: Greedy line packing, space distribution

Approach: Pack words greedily per line. Distribute extra spaces as evenly as possible.

TOPIC 3: TWO POINTERS & SLIDING WINDOW (Problems 61-80)

These patterns train your brain's ability to maintain dual attention tracks - a skill that strengthens the anterior cingulate cortex (conflict monitoring and resolution). You learn to mentally track two moving reference points simultaneously.

[Easy] #61. Remove Element

Origin: In-place filtering from early array processing.

Key Concepts: Two pointers (read/write)

Approach: Write pointer stays behind; copy non-target values forward.

[Easy] #62. Squares of Sorted Array

Origin: Merge-like operation accounting for negatives.

Key Concepts: Two pointers from ends, fill result from largest

Approach: Compare absolute values at both ends; place larger square at back of result.

[Easy] #63. Backspace String Compare

Origin: Text editor simulation with delete key.

Key Concepts: Reverse iteration with skip counter

Approach: Process from end; count backspaces, skip chars accordingly. Compare in O(1) space.

[Medium] #64. Sort Colors (Dutch National Flag)

Origin: Dijkstra's Dutch National Flag problem (1976).

Key Concepts: Three-way partition, three pointers (lo, mid, hi)

Approach: lo tracks 0-boundary, hi tracks 2-boundary, mid scans. Swap to correct region.

[Medium] #65. 3Sum Closest

Origin: Optimization variant of 3Sum from computational geometry.

Key Concepts: Sort + two pointers, track closest sum

Approach: Fix one element, two-pointer scan remaining. Update closest when |diff| smaller.

[Medium] #66. 4Sum

Origin: k-Sum generalization from combinatorial optimization.

Key Concepts: Sort + reduce to 3Sum then 2Sum

Approach: Two nested loops + two-pointer innermost. Prune with min/max checks.

[Medium] #67. Minimum Size Subarray Sum

Origin: Resource allocation: minimum contiguous resource block.

Key Concepts: Sliding window (variable size), shrink when sum >= target

Approach: Expand right until sum >= target. Shrink left while still valid. Track min length.

[Medium] #68. Fruit Into Baskets (Max 2 Types)

Origin: Inventory/resource constraint problem.

Key Concepts: Sliding window with at most 2 distinct, hash map

Approach: Maintain window with at most 2 fruit types. Shrink when third type enters.

[Medium] #69. Permutation in String

Origin: Substring anagram detection from security/pattern matching.

Key Concepts: Fixed-size sliding window, frequency match

Approach: Window of size s1.length. Slide across s2, compare frequency arrays.

[Medium] #70. Longest Substring with At Most K Distinct

Origin: Data stream analysis with bounded categories.

Key Concepts: Sliding window, hash map tracking distinct count

Approach: Expand right. When distinct > k, shrink left until <= k. Track max length.

[Hard] #71. Trapping Rain Water (Two Pointer)

Origin: Revisited with O(1) space using pointer technique.

Key Concepts: Two pointers, leftMax and rightMax tracking

Approach: Process from side with smaller max. Water determined by smaller boundary.

[Hard] #72. Subarrays with K Different Integers

Origin: Exact-count via at-most technique. Advanced counting.

Key Concepts: atMost(k) - atMost(k-1) transformation

Approach: Count subarrays with exactly k = atMost(k) - atMost(k-1). Two sliding windows.

[Hard] #73. Minimum Window Subsequence

Origin: Document search: find smallest window containing pattern as subsequence.

Key Concepts: Two-pointer forward scan then backward optimization

Approach: Find end by scanning forward. Then scan backward to minimize window start.

[Medium] #74. Interval List Intersections

Origin: Calendar scheduling: find overlapping free/busy times.

Key Concepts: Two pointers, one per list, merge-scan

Approach: Intersection = [max(starts), min(ends)]. Advance pointer with earlier end.

[Medium] #75. Boats to Save People

Origin: Bin packing variant from logistics and operations research.

Key Concepts: Sort + two pointers (lightest + heaviest)

Approach: Pair heaviest with lightest if combined <= limit. Otherwise heaviest rides alone.

[Medium] #76. Remove Duplicates II (At Most 2)

Origin: Data cleaning with bounded duplicates.

Key Concepts: Two pointers, count tracking

Approach: Write pointer allows at most 2 of same value. Compare with write[-2].

[Medium] #77. Max Consecutive Ones III

Origin: Signal processing: longest good segment with k error corrections.

Key Concepts: Sliding window, zero count tracking

Approach: Maintain window with at most k zeros. Shrink when zeros exceed k.

[Medium] #78. Longest Mountain in Array

Origin: Terrain analysis / geophysical signal processing.

Key Concepts: Two-pointer expand from peak or single scan

Approach: Find peaks (a[i-1] < a[i] > a[i+1]). Expand left and right from peak.

[Medium] #79. Partition Labels

Origin: Text segmentation with non-overlapping constraint.

Key Concepts: Last occurrence map + greedy interval merging

Approach: Track last index of each char. Extend current partition to cover all chars within.

[Hard] #80. Smallest Range Covering Elements from K Lists

Origin: Multi-source merge from distributed systems.

Key Concepts: Min-heap with one element per list, track max

Approach: Heap of k pointers. Range = [heap_min, global_max]. Advance min pointer.

PATTERN CHECKPOINT: You now have 3 core patterns in your neural toolkit - hash map lookups, two-pointer convergence, and sliding window. These patterns combine to solve ~40% of all interview

problems.

TOPIC 4: STACKS & QUEUES (Problems 81-105)

Stack thinking mirrors how your brain manages nested context (like nested function calls). Monotonic stacks train the brain to recognize 'next greater/smaller' patterns that appear everywhere from stock price analysis to histogram problems.

[Medium] #81. Min Stack

Origin: System design: O(1) min tracking. Origin: algorithm analysis tooling.

Key Concepts: Auxiliary stack or tuple (val, currentMin)

Approach: Store (value, current_min) at each level. Min is always top's second element.

[Medium] #82. Evaluate Reverse Polish Notation

Origin: HP calculators (1960s). Stack-based expression evaluation.

Key Concepts: Operand stack, operator triggers pop-compute-push

Approach: Push numbers. On operator, pop two, compute, push result.

[Medium] #83. Daily Temperatures

Origin: Weather/finance: 'how many days until warmer?'

Key Concepts: Monotonic decreasing stack of indices

Approach: Stack holds indices of unresolved days. Pop when current temp > stack top temp.

[Easy] #84. Next Greater Element I

Origin: Stock/signal analysis: next larger value.

Key Concepts: Monotonic stack + hash map for mapping

Approach: Process from right. Stack maintains decreasing sequence. Map each to its next greater.

[Medium] #85. Next Greater Element II (Circular)

Origin: Circular array variant using double-pass trick.

Key Concepts: Monotonic stack, iterate array twice (mod n)

Approach: Process 2n elements (index mod n). Same monotonic stack logic.

[Medium] #86. Asteroid Collision

Origin: Physics simulation. Stack models sequential impacts.

Key Concepts: Stack, process right-moving vs left-moving collisions

Approach: Push right-movers. Left-movers pop right-movers if larger. Handle ties.

[Medium] #87. Car Fleet

Origin: Traffic simulation: cars merging into fleets at destination.

Key Concepts: Sort by position, stack-based fleet merging

Approach: Sort by position desc. Stack of arrival times. Pop if arrives same time or later.

[Easy] #88. Valid Parentheses (Extended)

Origin: Compiler design: multi-type bracket matching.

Key Concepts: Stack with type checking on pop

Approach: Push open brackets. On close, check stack top matches type. Must end empty.

[Medium] #89. Generate Parentheses

Origin: Catalan number enumeration. Compiler/grammar generation.

Key Concepts: Backtracking with open/close count constraints

Approach: Add '(' if open < n. Add ')' if close < open. Base: length == 2n.

[Medium] #90. Simplify Path

Origin: Unix file system path normalization.

Key Concepts: Stack, split by '/', handle '.', '..', empty

Approach: Split path. Push directories. Pop for '..'. Ignore '.' and empty.

[Medium] #91. Decode String

Origin: Run-length encoding decoding. Data compression.

Key Concepts: Stack of (string, count) frames, or recursion

Approach: Push (current_string, count) when hitting '['. Pop and repeat on ']'.

[Medium] #92. Remove K Digits

Origin: Greedy digit removal for minimum number.

Key Concepts: Monotonic increasing stack, remove larger leading digits

Approach: Pop stack while top > current and k > 0. Push current. Remove leading zeros.

[Medium] #93. Online Stock Span

Origin: Financial analysis: consecutive days of lower-or-equal price.

Key Concepts: Monotonic stack storing (price, span) pairs

Approach: Pop all smaller-or-equal prices, accumulating their spans.

[Hard] #94. Largest Rectangle in Histogram

Origin: Computational geometry (Skiena). Image processing.

Key Concepts: Monotonic increasing stack of indices

Approach: Pop when height decreases. Width = current - stack_top - 1. Compute area.

[Hard] #95. Maximal Rectangle

Origin: Extends histogram to 2D. Image region detection.

Key Concepts: Row-by-row histogram heights + largest rectangle

Approach: Build height array per row. Apply histogram algorithm to each row.

[Hard] #96. Basic Calculator

Origin: Expression parser from compiler design (recursive descent).

Key Concepts: Stack for operands, handle +, -, and parentheses

Approach: Process char by char. Push/pop context on parentheses. Apply operators.

[Medium] #97. Basic Calculator II

Origin: Expression parser with precedence (* / before + -).

Key Concepts: Stack, defer + - until * / resolved

Approach: Evaluate * / immediately. Push results. Final sum of stack for + - .

[Easy] #98. Implement Queue using Stacks

Origin: Amortized analysis classic from algorithm theory.

Key Concepts: Two stacks: input and output, lazy transfer

Approach: Push to input stack. On pop/peek, transfer to output stack if output empty.

[Easy] #99. Implement Stack using Queues

Origin: Theoretical exercise in data structure simulation.

Key Concepts: One queue, rotate on push or pop

Approach: After push, rotate queue so new element is at front. Or rotate on pop.

[Medium] #100. Design Circular Queue

Origin: Buffered I/O, producer-consumer from OS design.

Key Concepts: Array with head/tail pointers, modular arithmetic

Approach: head and tail indices mod capacity. Track size for full/empty distinction.

[Hard] #101. Sliding Window Maximum

Origin: Revisited: monotonic deque for O(n) stream processing.

Key Concepts: Monotonic decreasing deque of indices

Approach: Remove expired indices. Remove smaller elements from back. Front is max.

[Hard] #102. Trapping Rain Water (Stack)

Origin: Stack-based approach to elevation/water problem.

Key Concepts: Monotonic stack, process bounded regions on pop

Approach: Pop when current > top. Water in bounded region = width * min(boundaries) - top height.

[Medium] #103. Remove All Adjacent Duplicates II

Origin: String compression / cleanup. Game elimination logic.

Key Concepts: Stack of (char, count) pairs

Approach: Push chars, increment count if same as top. Pop when count reaches k.

[Medium] #104. Score of Parentheses

Origin: Recursive structure scoring. Grammar value computation.

Key Concepts: Stack, () = 1, (A) = 2*A, AB = A+B

Approach: Track depth. () contributes 2^depth. Stack tracks nested scores.

[Hard] #105. Maximum Frequency Stack

Origin: LFU Cache variant. Design problem combining multiple structures.

Key Concepts: Stack per frequency + frequency map + max frequency

Approach: Push: increment freq, push to freq_stack[freq]. Pop: pop from max_freq stack.

TOPIC 5: LINKED LISTS (Problems 106-125)

Linked list problems train spatial reasoning and pointer manipulation - strengthening the parietal cortex. The ability to mentally track references and modify them builds skills that transfer to understanding databases, file systems, and any reference-based architecture.

[Easy] #106. Reverse Linked List

Origin: Fundamental operation from Lisp (1958). Tests pointer manipulation.

Key Concepts: Three pointers: prev, curr, next

Approach: Iteratively: save next, point curr to prev, advance prev and curr.

[Easy] #107. Merge Two Sorted Lists

Origin: Merge step of merge sort applied to lists.

Key Concepts: Dummy head, compare-and-append

Approach: Create dummy node. Compare heads of both lists, append smaller, advance.

[Easy] #108. Linked List Cycle

Origin: Floyd's cycle detection (1967). Memory leak detection origin.

Key Concepts: Slow/fast pointers (tortoise and hare)

Approach: Slow moves 1 step, fast moves 2. If they meet, cycle exists.

[Medium] #109. Linked List Cycle II (Find Start)

Origin: Extension of Floyd's: find WHERE cycle begins.

Key Concepts: Floyd's + reset one pointer to head

Approach: After meeting point, reset one to head. Move both one step until they meet at cycle start.

[Easy] #110. Middle of Linked List

Origin: Fast/slow pointer technique. Foundation for merge sort on lists.

Key Concepts: Slow/fast pointers

Approach: When fast reaches end, slow is at middle.

[Medium] #111. Remove Nth Node From End

Origin: One-pass deletion using lead/lag pointers.

Key Concepts: Two pointers with n-node gap

Approach: Advance fast n steps. Then advance both until fast reaches end. Delete at slow.

[Medium] #112. Palindrome Linked List

Origin: Combines multiple techniques: find middle, reverse, compare.

Key Concepts: Fast/slow + reverse second half + compare

Approach: Find middle, reverse second half, compare with first half, restore.

[Medium] #113. Reorder List (L0,Ln,L1,Ln-1...)

Origin: Interleaving pattern from parallel processing.

Key Concepts: Find middle + reverse second half + merge alternating

Approach: Split at middle, reverse second half, interleave-merge both halves.

[Medium] #114. Add Two Numbers

Origin: Big number addition. Origin: arbitrary-precision arithmetic.

Key Concepts: Carry-propagation, digit-by-digit sum

Approach: Sum digits + carry. Create new node for each digit. Propagate carry.

[Medium] #115. Add Two Numbers II

Origin: Most-significant-digit-first variant. Requires reversal or stack.

Key Concepts: Stack both lists, then add from top with carry

Approach: Push all digits onto stacks. Pop and add with carry. Build result backwards.

[Medium] #116. Flatten Multilevel Doubly Linked List

Origin: DOM tree flattening. Document structure linearization.

Key Concepts: DFS with pointer rewiring, stack for child branches

Approach: When child exists, insert child chain, reconnect next after chain end.

[Medium] #117. Copy List with Random Pointer

Origin: Object graph cloning. Origin: garbage collector mark-copy.

Key Concepts: Hash map (old->new) or interleaving technique

Approach: Interleave: insert clone after each original. Set random pointers. Separate lists.

[Medium] #118. Sort List (Merge Sort)

Origin: Efficient sorting on linked lists. O(n log n), O(1) space.

Key Concepts: Bottom-up merge sort, or top-down with fast/slow split

Approach: Find middle with fast/slow. Recursively sort halves. Merge sorted halves.

[Easy] #119. Intersection of Two Linked Lists

Origin: Finding shared memory references. Garbage collection analysis.

Key Concepts: Length difference or two-pointer convergence

Approach: Both pointers traverse own list then switch. Meet at intersection or null.

[Medium] #120. Swap Nodes in Pairs

Origin: Pointer manipulation practice. Link rewiring.

Key Concepts: Recursive or iterative pair swapping with prev tracking

Approach: Save pair, reverse their pointers, connect to prev and next pair.

[Hard] #121. Reverse Nodes in k-Group

Origin: Generalized group reversal. Batch processing metaphor.

Key Concepts: Count k nodes, reverse group, recursively handle rest

Approach: Check k nodes exist. Reverse k nodes. Connect to recursively processed remainder.

[Hard] #122. Merge K Sorted Lists

Origin: External merge sort from database systems. K-way merge.

Key Concepts: Min-heap of list heads, or divide-and-conquer merge

Approach: Min-heap: push all heads, pop min, push its next. D&C: pairwise merge until one.

[Medium] #123. LRU Cache

Origin: Operating system page replacement (Denning, 1968). Web caches.

Key Concepts: Hash map + doubly linked list for O(1) access and eviction

Approach: Map key to DLL node. On access: move to front. On insert when full: remove tail.

[Hard] #124. LFU Cache

Origin: Web proxy caches. Frequency-based eviction policy.

Key Concepts: Double hash map + frequency doubly linked lists

Approach: Map key to node. Map freq to DLL. Promote on access. Evict from min-freq list.

[Medium] #125. Design Linked List

Origin: Implementing the data structure itself. Systems programming.

Key Concepts: Node class, head/tail pointers, size tracking

Approach: Implement addAtHead, addAtTail, addAtIndex, deleteAtIndex, get.

TOPIC 6: BINARY TREES & BST (Problems 126-165)

Tree problems activate the brain's hierarchical reasoning systems. Recursion on trees parallels how the brain processes nested structures in language (subordinate clauses) and spatial reasoning (part-whole relationships). This topic causes the most neuroplastic growth in the medial frontal gyrus.

Binary Tree Fundamentals

[Easy] #126. Inorder Traversal

Origin: Tree walks from Knuth's 'Art of Programming' (1968).

Key Concepts: Recursion or iterative with stack

Approach: Left, Root, Right. Iterative: push all lefts, pop, process, go right.

[Easy] #127. Preorder Traversal

Origin: Used in serialization, expression tree printing.

Key Concepts: Recursion or stack (push right then left)

Approach: Root, Left, Right. Stack: push root, pop, push right child then left child.

[Easy] #128. Postorder Traversal

Origin: Used in deletion, expression evaluation.

Key Concepts: Recursion or two stacks / modified preorder reversal

Approach: Left, Right, Root. Can reverse modified preorder (Root, Right, Left).

[Medium] #129. Level Order Traversal (BFS)

Origin: Breadth-first from graph theory. Layer-by-layer processing.

Key Concepts: Queue, process level by level

Approach: Queue with level tracking. Process all nodes at current level, enqueue children.

[Easy] #130. Maximum Depth of Binary Tree

Origin: Tree measurement. Basis for balancing algorithms.

Key Concepts: Recursive: 1 + max(left_depth, right_depth)

Approach: Base: null returns 0. Recurse on children, take max, add 1.

[Easy] #131. Same Tree

Origin: Structural comparison. Used in parse tree equivalence checking.

Key Concepts: Simultaneous recursion on both trees

Approach: Both null: true. One null: false. Compare val, recurse left and right.

[Easy] #132. Invert Binary Tree

Origin: Famous for Homebrew creator's Google interview story.

Key Concepts: Recursive swap of left and right children

Approach: Swap left and right children. Recurse on both. Base: null returns null.

[Easy] #133. Symmetric Tree

Origin: Mirror comparison. Extension of 'same tree' concept.

Key Concepts: Recursive: compare left subtree with mirror of right

Approach: isMirror(left, right): compare vals, recurse (l.left,r.right) and (l.right,r.left).

[Easy] #134. Diameter of Binary Tree

Origin: Longest path in tree. Network diameter from graph theory.

Key Concepts: DFS tracking height, update global diameter

Approach: At each node: diameter through this node = leftHeight + rightHeight. Track global max.

[Easy] #135. Balanced Binary Tree

Origin: AVL tree prerequisite check. Database index validation.

Key Concepts: Recursive height check with early termination

Approach: Return height if balanced, -1 if not. Balanced: |leftH - rightH| <= 1 at every node.

Intermediate Tree Problems

[Easy] #136. Path Sum

Origin: Decision tree evaluation. Sum checking along root-to-leaf.

Key Concepts: DFS, subtract node value from target

Approach: Recurse subtracting current val. At leaf, check if remaining == 0.

[Medium] #137. Path Sum II (All Paths)

Origin: Enumerate all valid paths. Combinatorial tree search.

Key Concepts: DFS backtracking with path tracking

Approach: Track current path list. At leaf with sum==0, copy path to results. Backtrack.

[Medium] #138. Path Sum III (Any Start)

Origin: Subpath sum. Prefix sum technique on trees.

Key Concepts: Prefix sum hash map during DFS

Approach: Carry cumulative sum. Check if (cumSum - target) exists in prefix map. Backtrack map.

[Easy] #139. Subtree of Another Tree

Origin: Pattern matching on trees. Used in code clone detection.

Key Concepts: For each node, check isSameTree with target

Approach: At each node, check if subtree matches. Recurse on children.

[Medium] #140. Lowest Common Ancestor (BST)

Origin: Genealogy problem. Range query foundation.

Key Concepts: BST property: split point is LCA

Approach: If both < node, go left. Both > node, go right. Otherwise, current is LCA.

[Medium] #141. Lowest Common Ancestor (BT)

Origin: Genealogy in general trees. Extended from BST version.

Key Concepts: Post-order DFS, propagate findings upward

Approach: Recurse left and right. If both found, current is LCA. Else propagate found side.

[Medium] #142. Validate BST

Origin: Database index integrity check.

Key Concepts: Inorder traversal must be strictly increasing, or min/max range

Approach: Pass valid range (min, max) to each node. Update range as you descend.

[Medium] #143. Kth Smallest in BST

Origin: Order statistics on BSTs. Database SELECT with ORDER BY.

Key Concepts: Inorder traversal, count to k

Approach: Inorder gives sorted order. Count nodes visited, stop at k.

[Medium] #144. BST Iterator

Origin: Database cursor implementation. Lazy inorder traversal.

Key Concepts: Controlled inorder using explicit stack

Approach: Push all lefts initially. next(): pop, process, push all lefts of right child.

[Medium] #145. Construct BT from Preorder and Inorder

Origin: Unique tree reconstruction. Compiler parse tree building.

Key Concepts: Preorder gives root; inorder partitions left/right subtrees

Approach: Root = preorder[0]. Find root in inorder. Recurse on left/right partitions.

[Medium] #146. Construct BT from Inorder and Postorder

Origin: Mirror of preorder+inorder construction.

Key Concepts: Postorder gives root (last element); same partitioning logic

Approach: Root = postorder[-1]. Find in inorder. Recurse. Build right subtree first.

[Medium] #147. Flatten BT to Linked List

Origin: Tree serialization. Preorder threading.

Key Concepts: Reverse postorder, or Morris threading

Approach: Reverse postorder: process right, left, root. Set right = prev, left = null.

[Medium] #148. Populating Next Right Pointers

Origin: Level-linking for efficient BFS without queue.

Key Concepts: BFS or use existing next pointers as linked list

Approach: Use next pointers of current level to traverse and connect next level.

[Medium] #149. Binary Tree Right Side View

Origin: Visualization problem. Level-order last element.

Key Concepts: BFS (last of each level) or DFS (right-first, track depth)

Approach: DFS: visit right first. Add node if depth == result.size (first at this depth).

[Medium] #150. Count Complete Tree Nodes

Origin: Heap validation. Exploit complete tree structure for O(log<super>2</super>n).

Key Concepts: Compare left and right subtree heights

Approach: If left height == right height, left is perfect: 2^h - 1 + recurse right. Else recurse left.

Advanced Tree Problems

[Hard] #151. Serialize and Deserialize BT

Origin: Distributed systems: transmitting tree structures.

Key Concepts: Preorder with null markers, queue-based deserialization

Approach: Serialize: preorder with 'null' for missing children. Deserialize: recursive consume.

[Hard] #152. Binary Tree Maximum Path Sum

Origin: Critical path analysis. Network flow optimization.

Key Concepts: Post-order DFS, track global max, return single-branch max

Approach: At each node: maxGain = val + max(0, leftGain) + max(0, rightGain). Return single branch.

[Hard] #153. Vertical Order Traversal

Origin: Geographic plotting of trees. Column-based grouping.

Key Concepts: BFS with column index tracking, hash map column->nodes

Approach: Track (col, row, val) tuples. Group by column, sort by row then value.

[Hard] #154. Binary Tree Cameras

Origin: Minimum vertex cover variant. Surveillance optimization.

Key Concepts: Greedy post-order: 0=needs cover, 1=has camera, 2=covered

Approach: Post-order: if child needs cover, place camera. If child has camera, parent is covered.

[Medium] #155. Maximum Width of Binary Tree

Origin: Level analysis with position indexing.

Key Concepts: BFS with position tracking (2*pos, 2*pos+1)

Approach: Track leftmost and rightmost positions at each level. Width = right - left + 1.

[Medium] #156. All Nodes Distance K

Origin: Graph search on tree. Radial query.

Key Concepts: Convert tree to graph (parent pointers) + BFS from target

Approach: Build parent map via DFS. BFS from target node, expanding to left, right, parent.

[Medium] #157. Recover BST (Two Swapped Nodes)

Origin: Database index repair. Error correction.

Key Concepts: Inorder traversal, find two violations

Approach: Inorder traversal finds where order breaks. First violation's first and last violation's second are swapped.

[Medium] #158. House Robber III

Origin: Tree DP. Maximum independent set on tree.

Key Concepts: DFS returns (rob_this, skip_this) pair

Approach: Rob this = val + skip_left + skip_right. Skip this = max(rob,skip)_left + max(rob,skip)_right.

[Medium] #159. Trim BST

Origin: Range query pruning. Database index range deletion.

Key Concepts: Recursive: if out of range, return appropriate subtree

Approach: If val < low, return trimmed right. If val > high, return trimmed left. Else trim both.

[Medium] #160. Delete Node in BST

Origin: Hibbard deletion (1962). Maintaining BST invariant.

Key Concepts: Find successor (or predecessor) for node with two children

Approach: Leaf: remove. One child: bypass. Two children: replace with inorder successor, delete successor.

[Medium] #161. Morris Inorder Traversal

Origin: Morris (1979). O(1) space tree traversal using threading.

Key Concepts: Temporary threading via rightmost of left subtree

Approach: Find predecessor, thread its right to current. On second visit, restore and process.

[Medium] #162. Segment Tree (Range Sum Query)

Origin: Computational geometry (Bentley, 1977). Range queries.

Key Concepts: Build tree of range sums, update and query in O(log n)

Approach: Build: merge children. Update: traverse to leaf, propagate up. Query: split range.

[Medium] #163. Binary Indexed Tree (Fenwick)

Origin: Fenwick (1994). Elegant prefix sum updates.

Key Concepts: Use binary representation for parent-child relationships

Approach: Update: add to index, propagate via i += i & (-i). Query: sum via i -= i & (-i).

[Hard] #164. Count of Smaller Numbers After Self

Origin: Inversion count. Merge sort count or BST/BIT.

Key Concepts: Modified merge sort counting inversions, or BIT

Approach: During merge, count elements from right half merged before left elements.

[Hard] #165. Suffix Tree / Suffix Array Construction

Origin: Bioinformatics: genome pattern search. Ukkonen's algorithm.

Key Concepts: Suffix array with LCP. Or Ukkonen's O(n) suffix tree

Approach: Sort suffixes. Build LCP array for efficient substring queries.

TOPIC 7: GRAPHS (Problems 166-200)

Graph problems engage the most brain regions simultaneously: spatial reasoning (parietal), logical deduction (frontal), pattern recognition (visual cortex), and working memory (pallidum). They are the ultimate neuroplasticity workout for programmers.

[Medium] #166. Number of Islands

Origin: Geographical analysis. Connected component counting.

Key Concepts: DFS/BFS flood fill from each unvisited '1'

Approach: Scan grid. On finding '1', DFS/BFS to mark all connected '1's. Increment count.

[Medium] #167. Clone Graph

Origin: Network replication. Distributed systems node copying.

Key Concepts: BFS/DFS with hash map (old -> clone)

Approach: Map old nodes to clones. BFS: clone neighbors, connect, enqueue unvisited.

[Medium] #168. Course Schedule (Cycle Detection)

Origin: University prerequisite planning. Dependency resolution.

Key Concepts: Topological sort, DFS with 3-color marking

Approach: DFS: white=unvisited, grey=in-progress, black=done. Grey->Grey = cycle.

[Medium] #169. Course Schedule II (Topological Order)

Origin: Build order problem. Make file dependencies.

Key Concepts: Kahn's BFS (indegree) or DFS post-order reverse

Approach: Kahn's: enqueue zero-indegree nodes. Process, decrement neighbors' indegree.

[Medium] #170. Pacific Atlantic Water Flow

Origin: Hydrology simulation. Multi-source reachability.

Key Concepts: Reverse BFS/DFS from both oceans, find intersection

Approach: BFS from Pacific edges and Atlantic edges separately. Answer = intersection of reached cells.

[Medium] #171. Rotting Oranges

Origin: Epidemic simulation. Multi-source BFS for minimum time.

Key Concepts: Multi-source BFS from all initial rotten oranges

Approach: Enqueue all rotten. BFS level-by-level (each level = 1 minute). Check if fresh remain.

[Medium] #172. Walls and Gates

Origin: Shortest distance in grid. Multi-source BFS.

Key Concepts: BFS from all gates simultaneously

Approach: Enqueue all gates (0s). BFS outward, filling distances. First visit = shortest.

[Medium] #173. Surrounded Regions

Origin: Go game capture rule. Border-connected component analysis.

Key Concepts: DFS/BFS from border 'O's, mark as safe

Approach: Mark border-connected O's. Remaining O's are surrounded, flip to X.

[Medium] #174. Graph Valid Tree

Origin: Tree = connected acyclic graph. Structural validation.

Key Concepts: Union-Find or DFS: n-1 edges + connected

Approach: n nodes, n-1 edges, and connected = tree. Or: DFS detects no back edges.

[Medium] #175. Number of Connected Components

Origin: Network analysis. Social network clustering.

Key Concepts: Union-Find or DFS/BFS counting components

Approach: Initialize n components. Union edges. Count remaining distinct roots.

[Medium] #176. Redundant Connection

Origin: Finding the edge that creates a cycle. Network reliability.

Key Concepts: Union-Find: first edge where both endpoints already connected

Approach: Process edges in order. Union-Find: if find(u) == find(v), this edge is redundant.

[Hard] #177. Word Ladder

Origin: NLP: word transformation chains. Shannon's information theory.

Key Concepts: BFS on word graph (words differing by 1 char)

Approach: BFS from beginWord. Neighbors = words with one char changed that are in wordList.

[Hard] #178. Word Ladder II

Origin: All shortest transformation paths.

Key Concepts: BFS for distances + DFS backtracking for paths

Approach: BFS to build distance map. DFS from end to start following decreasing distances.

[Medium] #179. Minimum Height Trees

Origin: Tree centroid finding. Network optimization.

Key Concepts: Iterative leaf removal (topological peel)

Approach: Remove leaves repeatedly. Last 1-2 remaining nodes are centroids (roots of MHTs).

[Medium] #180. Cheapest Flights Within K Stops

Origin: Airline routing with hop constraint.

Key Concepts: Bellman-Ford (k+1 relaxations) or BFS with pruning

Approach: Bellman-Ford: k+1 iterations, relax all edges. Track best price to each city.

[Medium] #181. Network Delay Time

Origin: Dijkstra's algorithm (1959). Shortest path from source.

Key Concepts: Dijkstra with min-heap

Approach: Min-heap: (time, node). Process closest unvisited. Relax neighbors. Answer: max of all times.

[Hard] #182. Swim in Rising Water

Origin: Binary search + BFS, or Dijkstra-like priority BFS.

Key Concepts: Min-heap BFS (process lowest elevation first)

Approach: Priority queue: expand to neighbor with max(current_time, neighbor_elevation).

[Hard] #183. Alien Dictionary

Origin: Topological sort on character ordering from word list.

Key Concepts: Build DAG from adjacent word comparisons, topological sort

Approach: Compare adjacent words to find ordering edges. Topological sort the character graph.

[Hard] #184. Reconstruct Itinerary

Origin: Eulerian path problem. Origin: Euler's Konigsberg bridges (1736).

Key Concepts: Hierholzer's algorithm for Eulerian path

Approach: DFS from JFK. Visit lexicographically smallest airport. Post-order gives reverse path.

[Hard] #185. Critical Connections (Bridges)

Origin: Network reliability analysis. Tarjan's algorithm.

Key Concepts: Tarjan's bridge-finding with discovery and low-link values

Approach: DFS: track discovery time and lowest reachable ancestor. Edge (u,v) is bridge if low[v] > disc[u].

[Hard] #186. Strongly Connected Components

Origin: Social network analysis. Tarjan (1972) or Kosaraju.

Key Concepts: Kosaraju: two DFS passes (forward + reverse graph)

Approach: DFS1: finish-time order. DFS2: on reverse graph in reverse finish order. Each DFS2 tree = SCC.

[Medium] #187. Dijkstra's Shortest Path

Origin: Dijkstra (1959). Foundation of network routing (OSPF, GPS).

Key Concepts: Priority queue (min-heap) of (distance, node)

Approach: Greedily expand nearest unfinished node. Relax all neighbors. O((V+E) log V).

[Medium] #188. Bellman-Ford Algorithm

Origin: Bellman (1958). Handles negative edges.

Key Concepts: Relax all edges V-1 times

Approach: V-1 iterations, each relaxing all edges. Detects negative cycles on Vth iteration.

[Medium] #189. Floyd-Warshall (All Pairs)

Origin: Floyd (1962), Warshall (1962). All-pairs shortest paths.

Key Concepts: 3 nested loops: intermediate, source, destination

Approach: dp[i][j] = min(dp[i][j], dp[i][k] + dp[k][j]) for each intermediate k.

[Medium] #190. Prim's MST

Origin: Prim (1957). Network design (telecom, road, pipeline).

Key Concepts: Priority queue growing MST from arbitrary start

Approach: Start from any node. Add cheapest edge to unvisited node. Repeat until all visited.

[Medium] #191. Kruskal's MST

Origin: Kruskal (1956). Same goal as Prim's, different strategy.

Key Concepts: Sort edges by weight, Union-Find to avoid cycles

Approach: Sort edges. Add edge if it connects two different components (Union-Find check).

[Medium] #192. Topological Sort (DFS)

Origin: Task scheduling. Build systems. DAG processing.

Key Concepts: DFS post-order reversal

Approach: DFS from each unvisited node. Add to result in post-order reverse. Detects cycles.

[Medium] #193. Bipartite Graph Check

Origin: Job matching, 2-coloring. Konig's theorem applications.

Key Concepts: BFS/DFS 2-coloring

Approach: Color source. Alternate colors for neighbors. If same-color neighbor found, not bipartite.

[Medium] #194. Shortest Path in Binary Matrix

Origin: Grid pathfinding. Game AI movement.

Key Concepts: BFS on 8-directional grid

Approach: BFS from (0,0) to (n-1,n-1). 8 directions. First reach = shortest path.

[Medium] #195. Accounts Merge

Origin: Identity resolution. Email deduplication.

Key Concepts: Union-Find on email addresses per account

Approach: Union all emails within an account. Group by root. Attach to account name.

[Hard] #196. Making A Large Island

Origin: Grid optimization. Flip one 0 to maximize island.

Key Concepts: Label islands with BFS, precompute sizes, check each 0

Approach: DFS/BFS to label and size each island. For each 0, sum unique adjacent island sizes + 1.

[Hard] #197. Bus Routes

Origin: Public transit routing. BFS on route graph.

Key Concepts: BFS on routes (not stops), treating routes as nodes

Approach: Build route adjacency via shared stops. BFS from routes containing source to routes containing target.

[Medium] #198. Minimum Cost to Connect All Points

Origin: MST application: network wiring.

Key Concepts: Prim's or Kruskal's on complete graph

Approach: Compute all pairwise Manhattan distances. Apply Prim's with min-heap.

[Medium] #199. Graph Coloring (Backtracking)

Origin: Map coloring problem. NP-complete (general case).

Key Concepts: Backtracking with constraint propagation

Approach: Try colors 1..k for each node. Backtrack if adjacent nodes have same color.

[Hard] #200. Max Flow (Ford-Fulkerson)

Origin: Network flow theory. Ford & Fulkerson (1956). Resource allocation.

Key Concepts: BFS (Edmonds-Karp) for augmenting paths, residual graph

Approach: Find augmenting path via BFS. Push flow along path. Update residual capacities. Repeat.

TOPIC 8: DYNAMIC PROGRAMMING (Problems 201-255)

DP is the ultimate test of the medial frontal gyrus (deductive reasoning) and the prefrontal cortex (executive function). It requires holding multiple subproblems in working memory while seeing how they compose into the larger solution. This section will cause the most significant brain growth.

> *MINDFULNESS BEFORE DP: DP problems are where most people feel the most frustration. Before each DP problem, take 3 deep breaths. Remind yourself: every struggle with DP is literally building new neural pathways for abstract reasoning that will serve you for life.*

1D Dynamic Programming

[Easy] #201. Climbing Stairs

Origin: Fibonacci in disguise. Origin: combinatorial counting.

Key Concepts: DP[i] = DP[i-1] + DP[i-2], Fibonacci

Approach: Ways to reach step i = ways to reach (i-1) + ways to reach (i-2).

[Medium] #202. House Robber

Origin: Interval scheduling / maximum independent set on a path.

Key Concepts: DP[i] = max(DP[i-1], DP[i-2] + nums[i])

Approach: At each house: skip it (take prev best) or rob it (prev-prev best + current).

[Medium] #203. House Robber II (Circular)

Origin: Circular constraint reduces to two linear subproblems.

Key Concepts: Run House Robber on [0..n-2] and [1..n-1], take max

Approach: Can't rob both first and last. Solve two subproblems excluding each. Take max.

[Medium] #204. Coin Change

Origin: Classic DP from textbooks. Optimal change-making.

Key Concepts: DP[amount] = min coins to make amount

Approach: For each coin, dp[a] = min(dp[a], dp[a-coin] + 1). Initialize dp[0]=0, rest=INF.

[Medium] #205. Coin Change II (Count Ways)

Origin: Unbounded knapsack counting variant.

Key Concepts: DP[amount] with coin-by-coin iteration to avoid permutation counting

Approach: Process coins in outer loop to count combinations (not permutations).

[Medium] #206. Longest Increasing Subsequence

Origin: Patience sorting (card game). Dilworth's theorem application.

Key Concepts: DP O(n2) or greedy+binary search O(n log n)

Approach: DP: dp[i] = max dp[j]+1 for j less than i where a[j] less than a[i]. Binary search: maintain tails array.

[Medium] #207. Word Break

Origin: Text segmentation from NLP. Already covered but core DP.

Key Concepts: DP[i] = any j where DP[j] and s[j..i] in dict

Approach: dp[i] = true if some valid split point j exists.

[Medium] #208. Decode Ways

Origin: Encoding interpretation. Already covered but essential DP.

Key Concepts: DP[i] based on 1-digit and 2-digit decodings

Approach: dp[i] = dp[i-1] (if valid 1-digit) + dp[i-2] (if valid 2-digit).

[Medium] #209. Maximum Product Subarray

Origin: Extension of Kadane's. Handles negatives.

Key Concepts: Track both max and min product (negatives flip)

Approach: At each element: newMax = max(num, maxProd*num, minProd*num). Similarly for newMin.

[Medium] #210. Perfect Squares

Origin: Lagrange's four-square theorem (1770). BFS or DP.

Key Concepts: DP[n] = min(DP[n-k2]+1) for all k where k2<=n

Approach: For each number, try subtracting each perfect square. Take minimum.

[Medium] #211. Longest Palindromic Subsequence

Origin: Bioinformatics: RNA secondary structure prediction.

Key Concepts: 2D DP on subsequence, or LCS of s and reverse(s)

Approach: dp[i][j] = length of LPS in s[i..j]. If s[i]==s[j], dp[i+1][j-1]+2.

[Easy] #212. Min Cost Climbing Stairs

Origin: Weighted climbing stairs variant.

Key Concepts: DP[i] = cost[i] + min(DP[i-1], DP[i-2])

Approach: Minimum cost to reach step i = cost[i] plus cheaper of previous two steps.

[Medium] #213. Delete and Earn

Origin: Reduction to House Robber on value counts.

Key Concepts: Count values, apply House Robber on sorted unique values

Approach: Taking value v means deleting v-1 and v+1. Becomes House Robber on value axis.

[Medium] #214. Jump Game (DP/Greedy)

Origin: Reachability analysis. Can reduce from DP to greedy.

Key Concepts: Greedy max-reach tracking

Approach: Track farthest reachable. If current position > maxReach, return false.

[Medium] #215. Partition Equal Subset Sum

Origin: Subset sum (NP-complete in general). 0/1 Knapsack variant.

Key Concepts: DP on boolean reachability of sum/2

Approach: dp[j] = can we make sum j? For each num, update dp from right to left.

2D Dynamic Programming

[Medium] #216. Unique Paths

Origin: Grid traversal counting. Robot on grid. Combinatorics.

Key Concepts: DP[i][j] = DP[i-1][j] + DP[i][j-1]

Approach: Each cell reachable from top or left. Sum those counts.

[Medium] #217. Unique Paths II (Obstacles)

Origin: Grid with blocked cells.

Key Concepts: Same as Unique Paths but dp[obstacle] = 0

Approach: If cell is obstacle, dp = 0. Otherwise sum from top and left.

[Medium] #218. Minimum Path Sum

Origin: Grid optimization. Shortest weighted path.

Key Concepts: DP[i][j] = grid[i][j] + min(DP[i-1][j], DP[i][j-1])

Approach: Each cell = its value + minimum of top or left neighbor.

[Hard] #219. Edit Distance

Origin: Levenshtein (1965). Spell check, DNA alignment.

Key Concepts: DP[i][j] = min of insert, delete, replace operations

Approach: If chars match: dp[i-1][j-1]. Else: 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]).

[Medium] #220. Longest Common Subsequence

Origin: Bioinformatics: sequence alignment. Diff algorithms.

Key Concepts: DP[i][j]: LCS of first i of s1 and first j of s2

Approach: Match: dp[i-1][j-1]+1. No match: max(dp[i-1][j], dp[i][j-1]).

[Hard] #221. Distinct Subsequences

Origin: Pattern counting in strings. Regex matching variant.

Key Concepts: DP[i][j] = ways to form t[0..j] from s[0..i]

Approach: Match: dp[i-1][j-1] + dp[i-1][j]. No match: dp[i-1][j].

[Medium] #222. Interleaving String

Origin: String merging validation. Two-source interleaving.

Key Concepts: DP[i][j] = can s3[0..i+j] be formed by interleaving s1[0..i] and s2[0..j]

Approach: dp[i][j] = (s1[i]==s3[i+j] and dp[i-1][j]) or (s2[j]==s3[i+j] and dp[i][j-1]).

[Medium] #223. Maximal Square

Origin: Image processing: largest square of 1s.

Key Concepts: DP[i][j] = side length of largest square ending at (i,j)

Approach: dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1 if cell is '1'.

[Medium] #224. 0/1 Knapsack

Origin: Operations research classic (Dantzig, 1957). Resource allocation.

Key Concepts: DP[i][w] = max value with first i items and capacity w

Approach: Include item: dp[i-1][w-wt] + val. Exclude: dp[i-1][w]. Take max.

[Medium] #225. Target Sum (Subset Sum variant)

Origin: Count partitions with difference = target.

Key Concepts: Reduce to subset sum counting. dp on reachable sums

Approach: Find subsets summing to (total+target)/2. Count using 0/1 knapsack.

[Hard] #226. Regular Expression Matching

Origin: Thompson (1968). Pattern matching with . and *.

Key Concepts: DP[i][j] = s[0..i] matches p[0..j]

Approach: Handle '.': match any. Handle '*': zero occurrences or extend match.

[Hard] #227. Wildcard Matching

Origin: File glob patterns. DP with '?' and '*'.

Key Concepts: DP[i][j] = s[0..i] matches p[0..j]

Approach: '?': match any single. '*': dp[i-1][j] (extend) or dp[i][j-1] (skip *).

[Hard] #228. Burst Balloons

Origin: Interval DP. Origin: matrix chain multiplication paradigm.

Key Concepts: DP[i][j] = max coins from bursting balloons i..j

Approach: Try each balloon k as LAST burst in range i..j. dp[i][k-1] + dp[k+1][j] + nums[i-1]*nums[k]*nums[j+1].

[Hard] #229. Palindrome Partitioning II

Origin: Minimum cuts for palindrome substrings.

Key Concepts: DP for palindrome checking + DP for minimum cuts

Approach: dp_cut[i] = min cuts for s[0..i]. For each palindrome ending at i, dp_cut[i] = min(dp_cut[j-1]+1).

[Hard] #230. Egg Drop Problem

Origin: Operations research. Optimal testing with limited resources.

Key Concepts: DP[k][n] = min trials with k eggs and n floors

Approach: Binary search on optimal drop floor. dp[k][n] = 1 + min over x of max(dp[k-1][x-1], dp[k][n-x]).

Advanced DP Patterns

[Hard] #231. Longest Increasing Path in Matrix

Origin: Grid DP with memoization. Topological ordering on grid.

Key Concepts: DFS + memoization, process in topological order

Approach: memo[i][j] = longest path from (i,j). DFS to all 4 neighbors with greater value.

[Medium] #232. Best Time to Buy and Sell Stock with Cooldown

Origin: State machine DP. Financial trading model.

Key Concepts: States: hold, sold, rest. Transition between states

Approach: hold = max(hold, rest-price). sold = hold+price. rest = max(rest, sold).

[Medium] #233. Best Time Buy/Sell Stock with Transaction Fee

Origin: Trading with cost model.

Key Concepts: hold and cash states with fee deduction

Approach: cash = max(cash, hold+price-fee). hold = max(hold, cash-price).

[Hard] #234. Best Time Buy/Sell Stock III (At Most 2)

Origin: K-transaction generalization at k=2.

Key Concepts: State: day, transactions used, holding or not

Approach: Track buy1, sell1, buy2, sell2 values through array.

[Hard] #235. Best Time Buy/Sell Stock IV (At Most K)

Origin: General k-transaction stock problem.

Key Concepts: DP[k][i] = max profit with k transactions through day i

Approach: For each k: track maxDiff = max(dp[k-1][j] - price[j]). dp[k][i] = max(dp[k][i-1], price[i] + maxDiff).

[Hard] #236. Minimum Cost to Cut a Stick

Origin: Interval DP. Minimize total cutting cost.

Key Concepts: DP[i][j] = min cost to make all cuts between cuts[i] and cuts[j]

Approach: Try each cut point k between i and j. Cost = length + dp[i][k] + dp[k][j].

[Medium] #237. Stone Game Variants

Origin: Game theory DP. Minimax / alpha-beta.

Key Concepts: DP on subarray, alternating players

Approach: dp[i][j] = max score difference achievable by current player for stones[i..j].

[Medium] #238. Longest String Chain

Origin: Word chain by single character insertion.

Key Concepts: Sort by length, DP with predecessor check

Approach: Sort words by length. For each word, try removing each char; check if predecessor exists.

[Hard] #239. Arithmetic Slices II (Subsequences)

Origin: Counting arithmetic subsequences of length >= 3.

Key Concepts: DP with hash maps: dp[i][diff] = count ending at i with difference diff

Approach: For each pair (i,j), diff = a[i]-a[j]. dp[i][diff] += dp[j][diff] + 1.

[Hard] #240. Number of Longest Increasing Subsequences

Origin: Counting all LIS of maximum length.

Key Concepts: DP tracking both length and count at each position

Approach: dp_len[i] and dp_cnt[i]. When finding equal length, add counts.

[Medium] #241. Minimum ASCII Delete Sum

Origin: Edit distance variant with ASCII cost.

Key Concepts: DP[i][j] = min delete cost for s1[0..i] and s2[0..j]

Approach: Match: dp[i-1][j-1]. Delete from s1: dp[i-1][j]+ord(s1[i]). Delete from s2: dp[i][j-1]+ord(s2[j]).

[Medium] #242. Minimum Falling Path Sum

Origin: Grid optimization with 3-direction movement.

Key Concepts: DP[i][j] = matrix[i][j] + min of 3 cells above

Approach: dp[i][j] = val + min(dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1]).

[Medium] #243. Longest Arithmetic Subsequence

Origin: Find longest AP in array.

Key Concepts: DP[i][diff] = length of AP ending at i with common difference diff

Approach: For each pair (i,j): diff = a[i]-a[j]. dp[i][diff] = dp[j][diff] + 1.

[Hard] #244. Cherry Pickup (Two Robots)

Origin: Grid DP with two agents simultaneously.

Key Concepts: DP[row][col1][col2] = max cherries with both robots

Approach: Both robots move down one row. Try all 9 combinations of column moves. Share cell check.

[Hard] #245. Profitable Schemes

Origin: Multi-dimensional knapsack counting.

Key Concepts: DP[members_used][profit_so_far] = count of schemes

Approach: For each crime: update dp for using this crime's members and adding its profit.

[Hard] #246. Frog Jump

Origin: Variable jump length. State: (stone, last_jump).

Key Concepts: Hash map DP: dp[stone] = set of possible jump sizes to reach it

Approach: From each stone with jump k, can reach stone+k-1, stone+k, stone+k+1.

[Medium] #247. Paint House (3 Colors)

Origin: Graph coloring cost minimization on a path.

Key Concepts: dp[i][color] = min cost to paint house i with this color

Approach: dp[i][c] = cost[i][c] + min(dp[i-1][other colors]).

[Medium] #248. Ones and Zeroes

Origin: 2D knapsack: items cost (zeros, ones), maximize items.

Key Concepts: DP[m][n] = max items using m zeros and n ones

Approach: For each string, update dp: dp[i][j] = max(dp[i][j], dp[i-zeros][j-ones]+1).

[Hard] #249. Count Vowels Permutation

Origin: Finite automaton counting. State machine DP.

Key Concepts: DP per vowel state, transition rules

Approach: Each vowel has specific allowed predecessors. dp[i][v] = sum of dp[i-1][allowed predecessors of v].

[Hard] #250. Student Attendance Record II

Origin: String counting with constraints. State: (absences, late streak).

Key Concepts: DP[i][absences][consecutive_lates]

Approach: At each position, try Present, Absent (if A<1), Late (if L<2). Count valid strings.

[Hard] #251. Tallest Billboard

Origin: Meet-in-middle DP. Split into two equal-height rod groups.

Key Concepts: DP[diff] = max common height when difference between groups is diff

Approach: Process each rod: add to taller, shorter, or skip. Track by difference.

[Hard] #252. Strange Printer

Origin: Interval DP. Minimum print operations.

Key Concepts: DP[i][j] = min turns to print s[i..j]

Approach: dp[i][j] = dp[i][j-1]+1. But if s[k]==s[j] for some i<=k<j, dp[i][j] = min(dp[i][k]+dp[k+1][j-1]).

[Hard] #253. Number of Ways to Rearrange Sticks

Origin: Stirling numbers of the first kind. Combinatorics.

Key Concepts: DP[n][k] = ways to arrange n sticks seeing k from left

Approach: dp[n][k] = dp[n-1][k-1] + (n-1)*dp[n-1][k]. Shortest is visible, rest can go anywhere.

[Hard] #254. Minimum Difficulty of Job Schedule

Origin: Partition array into d contiguous groups, minimize sum of group maxes.

Key Concepts: DP[d][i] = min difficulty scheduling first i jobs in d days

Approach: dp[day][i] = min over j of (dp[day-1][j] + max(jobs[j+1..i])). Track running max.

[Hard] #255. K Inverse Pairs Array

Origin: Combinatorial DP. Counting permutations by inversions.

Key Concepts: DP[n][k] = permutations of 1..n with exactly k inverse pairs

Approach: dp[n][k] = dp[n][k-1] + dp[n-1][k] - dp[n-1][k-n]. Sliding window optimization.

TOPIC 9: HEAPS, GREEDY & BACKTRACKING (Problems 256-285)

[Medium] #256. Kth Largest Element

Origin: Order statistics. QuickSelect from Hoare (1961).

Key Concepts: Min-heap of size k, or QuickSelect

Approach: Min-heap of size k: push all, pop when size > k. Top is kth largest.

[Hard] #257. Find Median from Data Stream

Origin: Online statistics. Streaming data analysis.

Key Concepts: Two heaps: max-heap for lower half, min-heap for upper half

Approach: Balance sizes. Median = top of max-heap (or average of both tops).

[Hard] #258. Merge K Sorted Lists

Origin: External sort. Database merge. Priority queue application.

Key Concepts: Min-heap of k list heads

Approach: Push all heads. Pop minimum, push its next. Build result list.

[Medium] #259. Task Scheduler

Origin: CPU scheduling. OS design (Round Robin with cooldown).

Key Concepts: Greedy: most frequent task first, or math formula

Approach: Count frequencies. Slots = (maxFreq-1) * (n+1) + count of max-frequency tasks.

[Medium] #260. Reorganize String

Origin: Error-correcting codes. Non-adjacent same character.

Key Concepts: Max-heap of (count, char), alternate placement

Approach: Pop two most frequent chars alternately. If remaining freq > (n+1)/2, impossible.

[Medium] #261. Kth Smallest in Sorted Matrix

Origin: Multi-way merge on sorted structure.

Key Concepts: Min-heap or binary search on value range

Approach: Binary search: count elements <= mid. Adjust bounds to find kth position.

[Medium] #262. Top K Frequent Words

Origin: Search engine ranking. Information retrieval.

Key Concepts: Hash map + min-heap of size k (custom comparator)

Approach: Count frequencies. Min-heap with tiebreaking on lexicographic order.

[Medium] #263. Ugly Number II

Origin: Hamming's problem (1962). Numbers with only 2, 3, 5 as factors.

Key Concepts: Three pointers for factors 2, 3, 5, or min-heap

Approach: Maintain three pointers. Next ugly = min(ugly[p2]*2, ugly[p3]*3, ugly[p5]*5).

[Medium] #264. Jump Game (Greedy)

Origin: Greedy reachability analysis.

Key Concepts: Track maximum reachable index

Approach: At each position, update maxReach = max(maxReach, i + nums[i]). If i > maxReach, false.

[Medium] #265. Gas Station

Origin: Circular tour feasibility. Transportation logistics.

Key Concepts: Greedy: if total gas >= total cost, solution exists

Approach: Track surplus. If deficit at any point, start from next station.

[Easy] #266. Assign Cookies

Origin: Matching problem. Greedy smallest sufficient match.

Key Concepts: Sort children and cookies, match greedily

Approach: Sort both. For each child (ascending), find smallest sufficient cookie.

[Medium] #267. Non-overlapping Intervals

Origin: Activity/interval scheduling. Earliest-finish greedy.

Key Concepts: Sort by end time, greedily select non-overlapping

Approach: Sort by end. Keep interval if start >= previous end. Count removals.

[Medium] #268. Meeting Rooms II

Origin: Resource allocation. Minimum conference rooms needed.

Key Concepts: Sort by start time, min-heap of end times

Approach: Min-heap of ongoing meeting end times. If new meeting starts after earliest end, reuse room.

[Medium] #269. Minimum Number of Arrows

Origin: Interval piercing problem. Computational geometry.

Key Concepts: Sort by end, greedily shoot at earliest end

Approach: Sort balloons by end. Arrow at current end. Skip all balloons starting before arrow.

[Medium] #270. Queue Reconstruction by Height

Origin: Insertion sort variant. Greedy by height.

Key Concepts: Sort descending by height, then insert at position k

Approach: Tallest first: insert at index k[i]. Taller people don't see shorter, so position preserved.

[Easy] #271. Lemonade Change

Origin: Cash register simulation. Greedy change-making.

Key Concepts: Greedy: use largest bills first for change

Approach: Track $5 and $10 counts. For $20: prefer $10+$5, else three $5s.

[Medium] #272. Subsets

Origin: Power set enumeration. Foundation of combinatorics.

Key Concepts: Backtracking: include/exclude each element

Approach: For each element, recurse with and without it. Base: index == n.

[Medium] #273. Subsets II (With Duplicates)

Origin: Power set with duplicate handling.

Key Concepts: Sort + skip consecutive duplicates in backtracking

Approach: Sort. In backtracking loop, skip nums[i] == nums[i-1] when i > start.

[Medium] #274. Permutations

Origin: n! enumeration. Arrangement counting.

Key Concepts: Backtracking with used[] array or swap-based

Approach: Choose each unused element for current position. Recurse. Backtrack.

[Medium] #275. Permutations II (With Duplicates)

Origin: Permutations with duplicate pruning.

Key Concepts: Sort + skip same-value elements at same level

Approach: Sort. Skip if nums[i] == nums[i-1] and nums[i-1] not used (same-level duplicate).

[Medium] #276. Combination Sum

Origin: Unbounded selection. Coin change enumeration.

Key Concepts: Backtracking with reuse allowed, start index

Approach: At each level, try candidates from index onward (allows reuse). Prune if sum > target.

[Medium] #277. Combination Sum II

Origin: Bounded selection (each element used once).

Key Concepts: Sort + no reuse + skip duplicates at same level

Approach: Sort. Move to next index after choosing. Skip duplicates at same recursion level.

[Medium] #278. Letter Combinations of Phone Keypad

Origin: Telecom encoding. T9 predictive text.

Key Concepts: Backtracking through digit-to-letters mapping

Approach: For each digit, try each mapped letter. Recurse to next digit.

[Hard] #279. N-Queens

Origin: Chess puzzle (1848, Max Bezzel). Constraint satisfaction.

Key Concepts: Backtracking with column/diagonal tracking

Approach: Place queen per row. Check column, main diagonal, anti-diagonal conflicts. Backtrack.

[Hard] #280. Sudoku Solver

Origin: Constraint satisfaction from AI. Logic puzzle (1979).

Key Concepts: Backtracking with row/col/box constraint checking

Approach: Try 1-9 in each empty cell. Check row, column, 3x3 box constraints. Backtrack on failure.

[Medium] #281. Palindrome Partitioning

Origin: All palindromic decompositions. Backtracking + DP.

Key Concepts: Backtracking, check palindrome for each prefix

Approach: At each position, try all palindromic prefixes. Recurse on remainder.

[Medium] #282. Word Search

Origin: Grid DFS path finding. Pattern matching on 2D.

Key Concepts: DFS backtracking on grid with visited marking

Approach: For each cell matching word[0], DFS in 4 directions. Mark visited, backtrack.

[Hard] #283. Word Search II

Origin: Multiple pattern search on grid. Trie optimization.

Key Concepts: Trie of words + DFS backtracking on grid

Approach: Build trie from words. DFS from each cell, following trie edges. Prune exhausted branches.

[Medium] #284. Restore IP Addresses

Origin: Network address parsing. Constrained string partitioning.

Key Concepts: Backtracking: try 1-3 digit segments, validate 0-255

Approach: At each position, try taking 1, 2, or 3 digits. Validate range and leading zeros.

[Medium] #285. Combination Sum III

Origin: k numbers from 1-9 summing to n.

Key Concepts: Backtracking with size and sum constraints

Approach: Choose numbers 1-9 without repetition. Track count and remaining sum.

TOPIC 10: TRIES, INTERVALS, BIT MANIPULATION & MATH (Problems 286-300)

[Medium] #286. Implement Trie

Origin: Information retrieval (Fredkin, 1960). Auto-complete foundation.

Key Concepts: Node with children map and isEnd flag

Approach: Insert: traverse/create children. Search: traverse, check isEnd. StartsWith: traverse only.

[Medium] #287. Design Add and Search Words

Origin: Wildcard dictionary search. Regex on trie.

Key Concepts: Trie + DFS for '.' wildcard handling

Approach: On '.', recurse on all children. On letter, follow specific child.

[Hard] #288. Word Search II (Trie-based)

Origin: Multi-pattern grid search using trie.

Key Concepts: Build trie from words, DFS on grid following trie

Approach: Trie prunes DFS branches. Remove found words from trie (optimization).

[Medium] #289. Longest Word in Dictionary

Origin: Buildable words: each prefix must also be valid.

Key Concepts: Trie + BFS/DFS for longest buildable word

Approach: Insert all words. BFS/DFS through trie, only following nodes that are word endings.

[Medium] #290. Merge Intervals

Origin: Calendar/scheduling merge. Computational geometry.

Key Concepts: Sort by start, merge overlapping

Approach: Sort by start. If current overlaps previous, extend end. Else add new interval.

[Medium] #291. Insert Interval

Origin: Sorted interval list insertion and merge.

Key Concepts: Find overlap region, merge affected intervals

Approach: Add all before. Merge all overlapping. Add all after.

[Easy] #292. Meeting Rooms

Origin: Conflict detection. Simple overlap check.

Key Concepts: Sort by start, check adjacent overlaps

Approach: Sort. If any interval starts before previous ends, conflict exists.

[Easy] #293. Number of 1 Bits

Origin: Hardware popcount instruction. Hamming weight.

Key Concepts: n & (n-1) clears lowest set bit

Approach: Count iterations of n = n & (n-1) until n = 0.

[Easy] #294. Counting Bits

Origin: Compute popcount for all numbers 0 to n.

Key Concepts: DP: dp[i] = dp[i >> 1] + (i & 1)

Approach: Last bit contribution + previously computed result for right-shifted value.

[Easy] #295. Reverse Bits

Origin: Network byte order conversion. Hardware operation.

Key Concepts: Bit-by-bit extraction and placement

Approach: Extract LSB, place at MSB position. Shift input right, result left.

[Easy] #296. Missing Number

Origin: XOR trick or math (Gauss sum).

Key Concepts: XOR all indices with all values; or sum formula

Approach: Expected sum n*(n+1)/2 minus actual sum. Or XOR approach: extras cancel.

[Medium] #297. Pow(x, n)

Origin: Fast exponentiation. Origin: ancient Egyptian multiplication.

Key Concepts: Binary exponentiation: x^n = (x^(n/2))^2

Approach: If n even: recurse(x*x, n/2). If n odd: x * recurse(x, n-1). Handle negatives.

[Easy] #298. Happy Number

Origin: Recreational math. Cycle detection in sequence.

Key Concepts: Floyd's cycle detection or hash set

Approach: Sum of squared digits repeatedly. If reaches 1, happy. If cycle (no 1), not happy.

[Medium] #299. Count Primes (Sieve)

Origin: Eratosthenes (~240 BC). Oldest known algorithm.

Key Concepts: Sieve of Eratosthenes: mark multiples as composite

Approach: For each prime p, mark p*p, p*p+p, ... as not prime. Count unmarked.

[Medium] #300. Multiply Strings

Origin: Big number multiplication. Origin: long multiplication.

Key Concepts: Digit-by-digit multiplication with position tracking

Approach: result[i+j+1] += num1[i] * num2[j]. Handle carries. Build string.

PART 4: THE NEUROPLASTIC PROBLEM-SOLVER'S MANIFESTO

Daily Practice Protocol

Before Solving (5 min): Deep breathing. Set intention. Review yesterday's key insight. Your brain primes relevant neural circuits when you review before starting.

During Solving (60-90 min blocks): Work in focused intervals. When stuck, follow the 7-step protocol from Part 2. Write your thought process - this engages the medial frontal gyrus more deeply than just thinking.

After Solving (10 min): Reflect. What pattern did I use? Where else could this apply? What was the 'aha' moment? This reflection consolidates learning and builds transferable intuition.

Before Sleep: Briefly review the day's problems mentally. Your brain consolidates these memories during sleep, strengthening the neural pathways you built today.

The Growth Mindset Reminders

- â■¢ You are not 'bad at algorithms.' You are a brain that has not YET built those neural pathways.
- â■¢ Every minute of genuine struggle is neuroplastic change happening in real time.
- â■¢ The person who solved it in 5 minutes has seen that pattern 50 times before. You will too.
- â■¢ Confusion is not failure - it is your brain's signal that new connections are forming.
- â■¢ Sleep, exercise, and nutrition are not separate from DSA practice - they ARE part of it.
- â■¢ Progress in DSA is non-linear. Plateaus mean your brain is reorganizing before the next leap.
- â■¢ Comparing your Chapter 3 to someone else's Chapter 30 is neurologically meaningless.
- â■¢ The goal is not to memorize 300 solutions. It is to build 15-20 reusable thinking patterns.
- â■¢ When you solve a hard problem after hours of struggle, the dopamine reward is building your drive to tackle the next one.
- â■¢ You are not just learning algorithms. You are building a brain that thinks more clearly about everything.

"The brain that struggles with a hard problem today is not the same brain tomorrow. It is stronger, faster, and more connected. Trust the process."

This guide was created to be your companion through the journey. Return to Parts 1 and 2 whenever you need a reminder that your struggle has scientific meaning. Every problem you attempt - solved or not - is a gift to your future self's brain.